P151 : Simulation of Convertible Octacopter

Monthly Report August 2025

Mentors: Dr Santhakumar Mohan, Dr Vijay Muralidharan

Students: Parthiv P (Roll No: 132301026), Abhijith Sureshbabu (Roll No: 102301001)

Date: August 29, 2025

BACKGROUND

The initial attempt to design a quadcopter was unsuccessful. As a result, the focus was shifted to a more stable octacopter configuration, with the aim of enabling transformation mid-air.

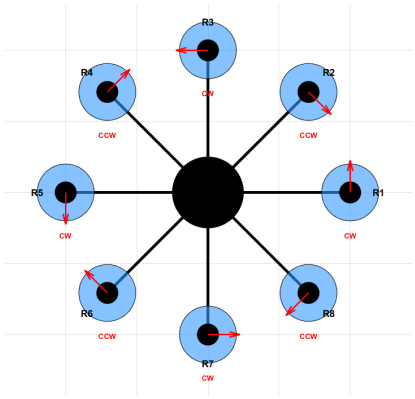


FIG. 1: Initial octacopter design layout.

MATHEMATICAL MODELLING

To support this, the team reviewed mathematical modeling approaches for quadcopters and extended the Newton–Euler method to suit the octacopter configuration.

$$\mathbf{M_B}\dot{\mathbf{v}} + \mathbf{C_B}(\mathbf{v})\mathbf{v} = \mathbf{g_B}(\xi) + \mathbf{o_B}(\mathbf{v}) + \mathbf{E_B}\omega^2, \tag{1}$$

is Newton's second law of motion. To calculate the acceleration, the equation can be rearranged as

$$\dot{\mathbf{v}} = {\mathbf{M_B}}^{-1}\left[-\mathbf{C_B}(\mathbf{v})\mathbf{v} + \mathbf{g_B}(\xi) + \mathbf{o_B}(\mathbf{v})\omega + \mathbf{E_B}\omega^2\right]. \tag{2}$$

where:

$$\mathbf{E_B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & b & 0 & b & 0 & b & 0 & b \\ b & 0 & b & 0 & b & 0 & b & 0 \\ -bl & \frac{-bl}{\sqrt{2}} & 0 & \frac{bl}{\sqrt{2}} & bl & \frac{bl}{\sqrt{2}} & 0 & \frac{-bl}{\sqrt{2}} \\ \frac{-bl}{\sqrt{2}} & -bl & \frac{-bl}{\sqrt{2}} & 0 & \frac{bl}{\sqrt{2}} & bl & \frac{bl}{\sqrt{2}} & 0 \\ d & -d & d & -d & d & -d & d & -d \end{bmatrix} \tag{3}$$

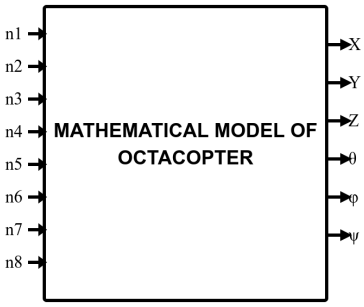


FIG. 2

SIMULATION WORK

A basic simulation of the octacopter was carried out to study how different rotor speeds influence maneuvering. For more details, see the simulation here: Matlab Simulation

RESULTS AND VISUALISATION

MATLAB simulations were developed to generate graphs of position and orientation versus time, providing a clearer understanding of drone motion.

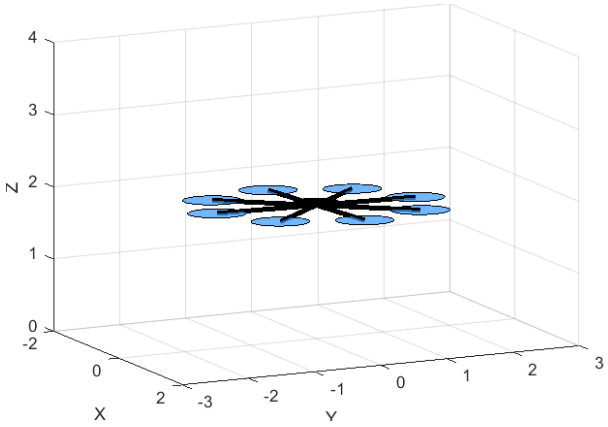


FIG. 3: MATLAB simulation showing realtime drone movement

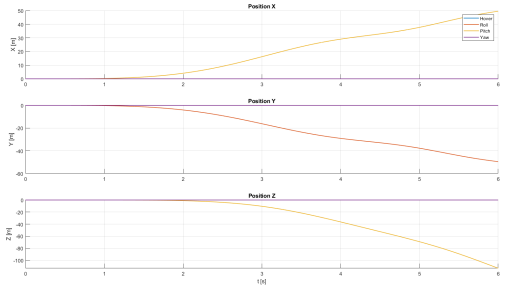


FIG. 4: Position v/s Time graph for Hover, Roll, Pitch, Yaw

FACULTY FEEDBACK AND APPROVAL

Dr. Santhakumar Mohan

The team has begun working on the basic simulation and has identified several important insights. Overall, the progress is satisfactory

30/8/25

Signature & Date

Dr. Vijay Muralidharan

Progress is satisfactory

M. Vijay

Signature & Date

Monthly Report September 2025

BACKGROUND

The project was updated from an octacopter to a quadcopter with 8-DOF actuation. Each motor rotates along its leg axis (X-Y plane), allowing lateral thrust without roll or pitch, improving maneuverability.

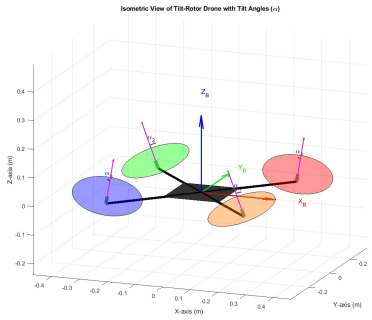


FIG. 1: Schematic Diagram of the Quadcopter

MATHEMATICAL MODELLING

Dynamics are formulated using **Newton–Euler equations**:

$$\begin{aligned} m\,\dot{\mathbf{v}} &= m\mathbf{g} + \sum_{i=1}^{4} \mathbf{F}_i^{\text{world}}, \\ \mathbf{I}\,\dot{\boldsymbol{\omega}} &= \boldsymbol{\tau}_{\text{body}} - \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}), \end{aligned} \tag{1}$$

where thrust vectors $\mathbf{F}_i$ are rotated to the world frame via the orientation matrix, and torques $\boldsymbol{\tau}_{\text{body}}$ arise from rotor positions, thrust, and drag.

Quaternions for Orientation

Quaternions avoid gimbal lock. Kinematics:

$$\dot{\mathbf{q}} = \frac{1}{2}\boldsymbol{\Omega}(\boldsymbol{\omega})\mathbf{q}, \quad \boldsymbol{\Omega}(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}. \tag{2}$$

Actuation Mapping

$$\mathbf{E}_B(\boldsymbol{\alpha}) = \frac{1}{\sqrt{2}} \begin{bmatrix} s_1 & s_2 & -s_3 & -s_4 \\ -s_1 & s_2 & s_3 & -s_4 \\ \sqrt{2}c_1 & \sqrt{2}c_2 & \sqrt{2}c_3 & \sqrt{2}c_4 \\ -\sqrt{2}lc_1 + ds_1 & -\sqrt{2}lc_2 - ds_2 & \sqrt{2}lc_3 - ds_3 & \sqrt{2}lc_4 + ds_4 \\ \sqrt{2}lc_1 - ds_1 & -\sqrt{2}lc_2 - ds_2 & -\sqrt{2}lc_3 + ds_3 & \sqrt{2}lc_4 + ds_4 \\ -2ls_1 + \sqrt{2}dc_1 & -\sqrt{2}dc_2 & \sqrt{2}dc_3 & 2ls_4 - \sqrt{2}dc_4 \end{bmatrix} \tag{3}$$

This maps the individual rotor thrusts (T_1, T_2, T_3, T_4) to the resultant forces and torques on the drone's body. Where:

- $s_i = \sin(\alpha_i)$ and $c_i = \cos(\alpha_i)$ for each rotor i.
- l is the chassis arm length (e.g., 0.25 m).
- d is the rotor drag coefficient (e.g., 0.02).

SIMULATION WORK

- A MATLAB simulation was developed to study quadcopter motion under varying rotor thrust and tilt angles, using an open-loop model with constant inputs.
- In practice, thrust depends on rotor speed, while tilt angles follow first-order dynamics rather than changing instantaneously.
- Each rotor tilts about its chassis arm (at 45° to the body X–Y axes), with tilt angle α measured from the Z-axis, giving $F_x, F_y \propto \sin\alpha$ and $F_z \propto \cos\alpha$.

For simulation files: MATLAB Simulation Folder

RESULTS AND VISUALISATION

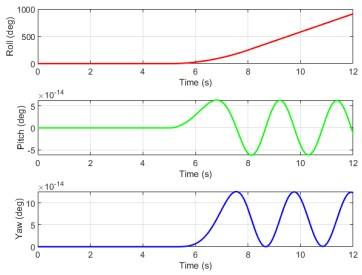


FIG. 2: Variation of Angles

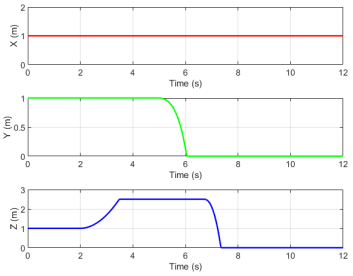


FIG. 3: Position v/s Time graph

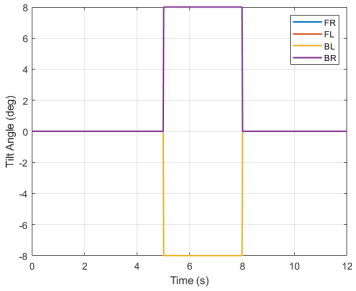


FIG. 4: Tilt Angle v/s Time

- **Position vs. Time**: X, Y, Z trajectories showing hover, ascent, lateral motion, and descent.
- **Orientation**: Roll, pitch, yaw remain stable during lateral translation.
- **Rotor Tilts**: α_i showing coordinated lateral motion.

Lateral motion is achieved without rolling the vehicle, demonstrating the effect of tilting rotors along the leg axes.

FACULTY FEEDBACK AND APPROVAL

Dr. Santhakumar Mohan

The progress of the team is good and satisfactory. Hopefully, in the coming month, full-fledged simulations will be performed.

30/9/25

Signature & Date

Dr. Vijay Muralidharan

Progress is satisfactory

30-9-2025

Signature & Date

Monthly Report October 2025

BACKGROUND

With the tilt-rotor quadcopter model finalized, October's focus shifted from open-loop simulation to active closed-loop control. We have begun implementing a foundational Proportional-Derivative (PD) controller to achieve stable attitude (roll, pitch, yaw) and position (X, Y, Z) tracking. This PD controller will serve as the baseline for evaluating more advanced control strategies.

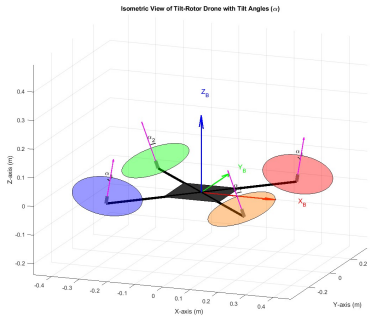


FIG. 1: Schematic Diagram of the Quadcopter

CONTROL MODELLING

This month's work focused on the development and testing of Proportional-Derivative (PD) control strategies, which form the foundation of the drone's stabilization and tracking capabilities.

- **Baseline Controller Development:** A foundational PD control system was first designed for a standard, non-tilting quadcopter. This system consisted of two independent PD loops:
  - **PD for Altitude:** This controller manages vertical position. It calculates the necessary total thrust by measuring the **Proportional** error (how far the drone is from its target height) and the **Derivative** error (how fast it is moving toward or away from that height).
  - **PD for Attitude:** A second PD controller stabilizes the drone's body. It calculates the required corrective **torques** by measuring the **Proportional** error (how far the drone is tilted from its target angle) and the **Derivative** error (how fast it is rotating).
- **Tilt-Rotor Controller Development:** The control logic was then adapted for the 8-DOF tilt-rotor model, focusing specifically on attitude tracking.
  - In this configuration, a **PD controller** is responsible for managing the drone's orientation (roll, pitch, and yaw).
  - The controller measures the **Proportional** error (the difference between the desired angle and the actual angle) and the **Derivative** error (the body's angular velocity).
  - Based on these errors, the PD controller computes the necessary stabilizing **torques**. This allows the drone to actively follow a commanded orientation and resist external disturbances, such as a simulated wind gust.

SIMULATION WORK

- A MATLAB simulation environment was developed to implement and test closed-loop **PD control strategies**, advancing from the previous open-loop analysis.

- A baseline controller was first validated on a standard, non-tilting quadcopter model. This simulation successfully demonstrated simultaneous **altitude tracking** (holding a target height) and **attitude stabilization** (maintaining a level hover).

- A second, more advanced simulation was created for the **8-DOF tilt-rotor model**. This test focused specifically on PD attitude control, locking the drone's position to analyze its ability to track a desired pitch command and reject simulated wind disturbances.

- The tilt angles α are actively controlled using a hybrid PD strategy that maps attitude errors to differential tilt commands. Independent PD controllers for roll, pitch, and yaw generate commands allocated to individual rotors via a mixing matrix, creating control torques through tilted thrust vectors. This direct superposition is valid under the small-angle assumption, which linearizes the torque-tilt relationship and eliminates coupling effects between axes.

For simulation files: MATLAB Simulation Folder

RESULTS AND VISUALISATION

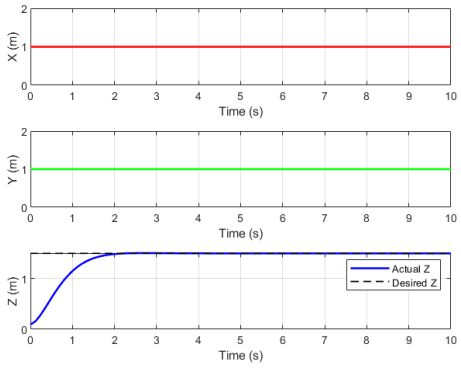


FIG. 2: Position vs Time Altitude

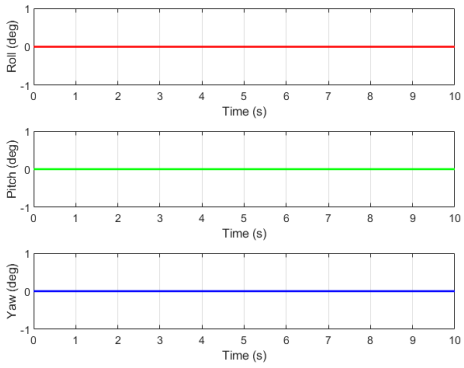


FIG. 3: Variation of Angles Altitude

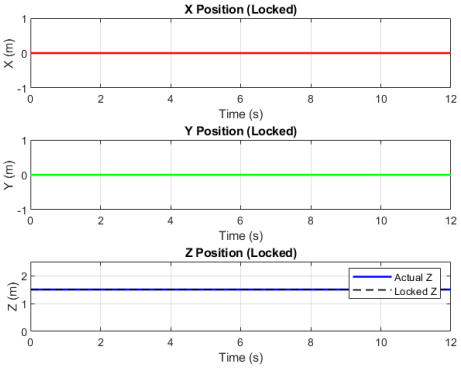


FIG. 4: Position vs Time Attitude

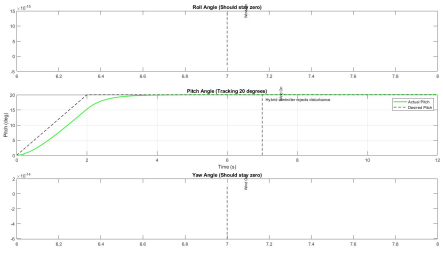
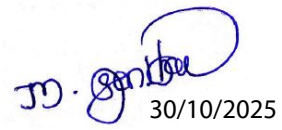


FIG. 5: Variation of Angles Attitude

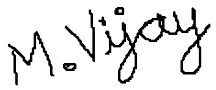
FACULTY FEEDBACK AND APPROVAL

Dr. Santhakumar Mohan
The progress is satisfactory and needs to perform detailed simulations before end semester.



Signature & Date

Dr. Vijay Muralidharan
Progress is satisfactory



Signature & Date

P151 : Simulation of Convertible Octacopter

Final Report November 2025

Mentors: Dr Santhakumar Mohan, Dr Vijay Muralidharan

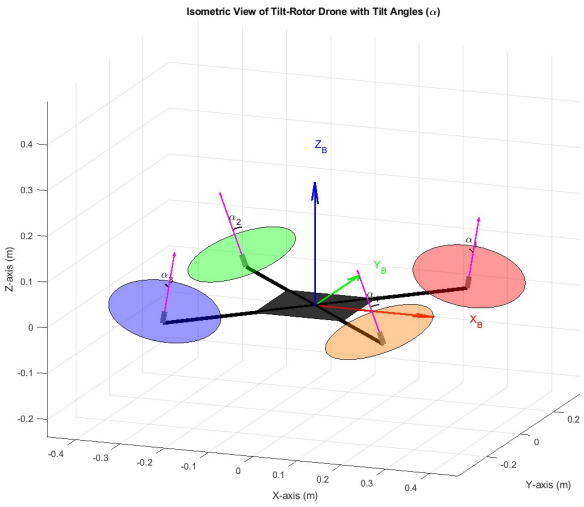
Students: Parthiv P (Roll No: 132301026), Abhijith Sureshbabu (Roll No: 102301001)

Date: November 29, 2025

ABSTRACT

This project explores the design, modeling, and control of advanced multi-rotor configurations. Initially exploring a convertible octacopter for mid-air transformation, the project evolved into a Quadcopter with 8-Degrees of Freedom (DOF) actuation. This configuration utilizes tilt-rotors to enable lateral thrust without altering pitch or roll. The project successfully established mathematical models using Newton-Euler dynamics and Quaternion kinematics, followed by the implementation of a closed-loop Proportional-Derivative (PD) control system to ensure stable altitude and attitude tracking.

MATHEMATICAL MODELLING



We define two reference frames:

- **World Frame (W)**: Inertial frame, ENU (East-North-Up) convention with gravity acting in $-Z_W$.
- **Body Frame (B)**: Fixed to the vehicle center of mass.

The system state $\mathbf{x} \in \mathbb{R}^{13}$ is defined as:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{q} \\ \boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} p_x, p_y, p_z, v_x, v_y, v_z, q_w, q_x, q_y, q_z, \omega_x, \omega_y, \omega_z \end{bmatrix}^T \tag{1}$$

Where:

- $\mathbf{p}$: Position in World Frame.
- $\mathbf{v}$: Velocity in World Frame.
- $\mathbf{q}$: Unit quaternion representing rotation from Body to World.
- $\boldsymbol{\omega}$: Angular velocity in Body Frame.

ACTUATOR DYNAMICS

The control inputs vector $\mathbf{u}$ consists of 4 motor thrusts (T_i) and 4 tilt angles (α_i):

$$\mathbf{u} = [T_1, T_2, T_3, T_4, \alpha_1, \alpha_2, \alpha_3, \alpha_4]^T \tag{2}$$

Rotor Tilt Transformation

For each motor i, the thrust acts initially along the motor's local z-axis. The motor can tilt around an axis $\mathbf{a}_i$ (the arm direction). The rotation matrix $\mathbf{R}_{tilt,i}$ for the tilt angle α_i is calculated using the Axis-Angle (Rodrigues') formula:

$$\mathbf{R}_{tilt,i} = \mathbf{I} + \sin(\alpha_i)[\mathbf{a}_i]_\times + (1 - \cos(\alpha_i))[\mathbf{a}_i]_\times^2 \tag{3}$$

Where $[\mathbf{a}_i]_\times$ is the skew-symmetric matrix of the tilt axis unit vector.

The force vector produced by motor i expressed in the **Body Frame** is:

$$\mathbf{F}_{b,i} = \mathbf{R}_{tilt,i} \begin{bmatrix} 0 \\ 0 \\ T_i \end{bmatrix} \tag{4}$$

FORCE AND TORQUE ANALYSIS

Torques in Body Frame

The total torque $\boldsymbol{\tau}_b$ acting on the body consists of the moment generated by the thrust and the reaction torque (drag torque) from the propeller.

1. **Thrust Torque:** Generated by the lever arm $\mathbf{r}_i$ from the CoM to the motor.

$$\boldsymbol{\tau}_{T,i} = \mathbf{r}_i \times \mathbf{F}_{b,i} \tag{5}$$

2. **Reaction Torque:** This torque acts opposite to the propeller rotation direction $\delta_i \in \{1, -1\}$. Crucially, in a tilt-rotor, this torque aligns with the *tilted* rotor axis (the 3rd column of $\mathbf{R}_{tilt,i}$).

$$\boldsymbol{\tau}_{R,i} = C_\tau \cdot \delta_i \cdot (\mathbf{R}_{tilt,i}\mathbf{e}_3) \cdot T_i \tag{6}$$

Where C_τ is the torque coefficient (`p.TorqueC`).

The total body torque is:

$$\boldsymbol{\tau}_b = \sum_{i=1}^{4} (\boldsymbol{\tau}_{T,i} + \boldsymbol{\tau}_{R,i}) \tag{7}$$

Forces in World Frame

The orientation of the drone is given by the rotation matrix $\mathbf{R}_{WB}(\mathbf{q})$ derived from the quaternion state. The total force in the World Frame is the sum of rotated motor forces and gravity:

$$\mathbf{F}_{total} = \left(\sum_{i=1}^{4} \mathbf{R}_{WB}(\mathbf{q})\mathbf{F}_{b,i} \right) + \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} \tag{8}$$

EQUATIONS OF MOTION

The system dynamics are governed by the Newton-Euler equations.

Translational Dynamics

$$\dot{\mathbf{p}} = \mathbf{v} \tag{9}$$

$$\dot{\mathbf{v}} = \frac{1}{m}\mathbf{F}_{total} \tag{10}$$

Rotational Dynamics

Using Euler's equations for rigid body dynamics, where $\mathbf{J}$ is the inertia matrix:

$$\dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}\left(\boldsymbol{\tau}_b - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})\right) \tag{11}$$

Quaternion Kinematics

The rate of change of the orientation quaternion is related to the angular velocity:

$$\dot{\mathbf{q}} = \frac{1}{2}\Omega(\boldsymbol{\omega})\mathbf{q} \tag{12}$$

Where $\Omega(\boldsymbol{\omega})$ is the skew-symmetric matrix representation:

$$\Omega(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \tag{13}$$

SYSTEM GEOMETRY

Rotor Positions

The quadcopter consists of four rotors. The position of the i-th rotor hub relative to the center of mass, denoted as $\mathbf{r}_i$, is defined in the Body Frame.

$$\mathbf{r}_1 = \begin{bmatrix} L \\ L \\ 0 \end{bmatrix}, \quad \mathbf{r}_2 = \begin{bmatrix} -L \\ L \\ 0 \end{bmatrix}, \quad \mathbf{r}_3 = \begin{bmatrix} -L \\ -L \\ 0 \end{bmatrix}, \quad \mathbf{r}_4 = \begin{bmatrix} L \\ -L \\ 0 \end{bmatrix} \tag{14}$$

Tilt Axes Definition

In this specific model, the tilt axes are **aligned with the arms**. This means the rotors tilt laterally relative to the arm, not forward/backward relative to the fuselage.

The tilt axis unit vector $\mathbf{a}_i$ for the i-th rotor is the normalized position vector:

$$\mathbf{a}_i = \frac{\mathbf{r}_i}{\|\mathbf{r}_i\|} = \frac{1}{\sqrt{2}}\begin{bmatrix} \pm 1 \\ \pm 1 \\ 0 \end{bmatrix} \tag{15}$$

Consequently, when a tilt angle α_i is applied, the thrust vector rotates around the arm, creating a lateral force component in the body frame.

CONTROL ALLOCATION (MIXING MATRIX)

The control allocation maps the desired virtual control efforts (Total Thrust F_z and Moments $\mathbf{M} = [M_x, M_y, M_z]^T$) to the individual motor thrusts T_i.

The Mixing Equation

The relationship is linear and defined by the mixing matrix $\mathbf{\Gamma}$:

$$\begin{bmatrix} F_{z,des} \\ M_{x,des} \\ M_{y,des} \\ M_{z,des} \end{bmatrix} = \mathbf{\Gamma} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \tag{16}$$

Matrix Structure

Based on the code implementation, the mixing matrix $\mathbf{\Gamma}$ is constructed as follows:

$$\mathbf{\Gamma} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ L & L & -L & -L \\ -L & L & L & -L \\ C_\tau & -C_\tau & C_\tau & -C_\tau \end{bmatrix} \tag{17}$$

- **Row 1 (Thrust):** Sum of all motor thrusts.
- **Row 2 (Roll M_x):** Generated by differential thrust based on the y-distance. Note that rotors 1 and 2 ($y > 0$) contribute positively to roll (assuming standard right-hand rule orientation).
- **Row 3 (Pitch M_y):** Generated by differential thrust based on the x-distance.
- **Row 4 (Yaw M_z):** Generated by reaction torques. C_τ is the torque coefficient (`p.TorqueC = 0.02`). The signs $[1, -1, 1, -1]$ correspond to the rotor rotation directions.

Inverse Allocation

To find the required motor thrusts for a desired set of forces and moments, the system solves the inverse problem:

$$\begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \mathbf{\Gamma}^{-1} \begin{bmatrix} F_{z,des}/\cos(\bar{\alpha}) \\ M_{x,cmd} \\ M_{y,cmd} \\ M_{z,cmd} \end{bmatrix} \tag{18}$$

Note: The code applies a correction factor $\cos(\bar{\alpha})$ *to the total thrust request to account for efficiency loss due to the average rotor tilt.*

ROTOR TILT CONTROL ALLOCATION

Unlike the motor rotational speeds, which are determined by the PID torque controller, the rotor tilt angles α_i are computed using a geometric projection method based on the current body attitude. This secondary control loop provides lateral force compensation.

Attitude Correction Vector

First, a correction vector $\mathbf{v}_{corr}$ is defined in the Body Frame to capture the lateral components of the drone's orientation (pitch θ and roll ϕ). This vector represents the direction of the gravitational slip relative to the body z-axis:

$$\mathbf{v}_{corr} = \begin{bmatrix} \sin(\theta) \\ -\sin(\phi) \\ 0 \end{bmatrix} \tag{19}$$

Projection onto Tilt Axes

The correction vector is projected onto the normalized tilt axis $\mathbf{u}_i$ of each rotor i. The tilt axis unit vector is defined by the arm direction:

$$p_i = \mathbf{v}_{corr} \cdot \mathbf{u}_i \tag{20}$$

Tilt Control Law

The commanded tilt angle $\alpha_{cmd,i}$ is proportional to this projection, scaled by a gain factor $k_{tilt} = 0.8$ (tuned for smooth response) and inverted to counteract the slip:

$$\alpha_{cmd,i} = -k_{tilt} \cdot p_i = -0.8 \cdot (\sin(\theta)u_{i,x} - \sin(\phi)u_{i,y}) \tag{21}$$

Saturation

Finally, the calculated tilt command is saturated to the mechanical limits of the servo mechanism ($\alpha_{max} = 15°$):

$$\alpha_i = \begin{cases} \alpha_{max} & \text{if } \alpha_{cmd,i} > \alpha_{max} \\ -\alpha_{max} & \text{if } \alpha_{cmd,i} < -\alpha_{max} \\ \alpha_{cmd,i} & \text{otherwise} \end{cases} \tag{22}$$

CONTROL STRATEGY

The control strategy implemented in the provided MATLAB simulation utilizes a **Cascaded Control Architecture** with **Time-Scale Separation**. This design decouples the translational dynamics (position) from the rotational dynamics (attitude) based on their response speeds.

- **Inner Loop (Attitude):** Runs at a high frequency (**200** Hz) to stabilize the fast rotational dynamics.
- **Outer Loop (Position):** Runs at a lower frequency (**25** Hz) to handle trajectory tracking.

This separation ensures that the inner loop settles quickly enough to be treated as a static gain by the outer loop (Bandwidth ratio $\approx 8:1$).

I. ANALYTIC POLE PLACEMENT TUNING

Instead of heuristic tuning (trial and error), the Proportional (K_p) and Derivative (K_d) gains are derived mathematically using *Analytic Pole Placement*. This method maps desired performance metrics—Natural Frequency (ω_n) and Damping Ratio (ζ)—to specific gain values based on the standard second-order characteristic equation:

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0 \tag{23}$$

Outer Loop: Position Control

The translational dynamics are modeled as a double integrator system normalized by mass ($\ddot{x} = u$). Under PD control, the closed-loop error dynamics are:

$$\ddot{e} + K_{d\_pos}\dot{e} + K_{p\_pos}e = 0 \tag{24}$$

By matching coefficients with the standard form, the gains are derived as:

$$K_{p\_pos} = \omega_n^2 \tag{25}$$
$$K_{d\_pos} = 2\zeta\omega_n \tag{26}$$

Implemented Values: The design targets a slightly underdamped response ($\zeta = 0.8$) for agility and a moderate bandwidth ($\omega_n = 1.5$ rad/s) for smooth motion.

- $K_{p\_pos} = 1.5^2 = \mathbf{2.25}$
- $K_{d\_pos} = 2(0.8)(1.5) = \mathbf{2.4}$

Inner Loop: Attitude Control

The rotational dynamics include the Moment of Inertia (I). The equation of motion is $\tau = I\ddot{\theta}$. The closed-loop characteristic equation becomes:

$$s^2 + \frac{K_{d\_att}}{I}s + \frac{K_{p\_att}}{I} = 0 \tag{27}$$

Matching the coefficients:

$$\frac{K_{p\_att}}{I} = \omega_n^2 \implies K_{p\_att} = I \cdot \omega_n^2 \tag{28}$$
$$\frac{K_{d\_att}}{I} = 2\zeta\omega_n \implies K_{d\_att} = I \cdot 2\zeta\omega_n \tag{29}$$

Implemented Values: The design targets a Critically Damped response ($\zeta = 1.0$) to prevent overshoot and a high bandwidth ($\omega_n = 12.0$ rad/s). Using $I_{xx} = 0.03$ kg $\cdot$ m^2:

- $K_{p\_att} = 0.03 \times 12.0^2 = \mathbf{4.32}$
- $K_{d\_att} = 0.03 \times 2(1.0)(12.0) = \mathbf{0.72}$

II. POSITION TRACKING (OUTER LOOP LOGIC)

The outer loop functions as a "Virtual Controller" that converts position errors into desired attitude angles.

1. **Error Calculation:** Computes the difference between the desired trajectory point (x_{des}, v_{des}) and current state.
2. **PID Law (Acceleration Command):**

$$\mathbf{a}_{cmd} = \mathbf{a}_{ff} + K_p(\mathbf{x}_{des} - \mathbf{x}) + K_d(\mathbf{v}_{des} - \mathbf{v}) \tag{30}$$

3. **Thrust Vectoring:** The total required force vector in the world frame is calculated to overcome gravity and provide the command acceleration:

$$\mathbf{F}_{des} = m(\mathbf{a}_{cmd} + g\hat{\mathbf{z}}) \tag{31}$$

4. **Attitude Conversion:** The controller converts the lateral components of $\mathbf{F}_{des}$ into Roll (ϕ) and Pitch (θ) commands. The code uses a hyperbolic tangent function (tanh) to smooth and saturate these commands:

$$\theta_{des} \propto \tanh(-Error_X)$$
$$\phi_{des} \propto \tanh(Error_Y)$$

III. ATTITUDE CONTROL (INNER LOOP LOGIC)

The inner loop executes the orientation commands by generating physical torques and rotor tilts.

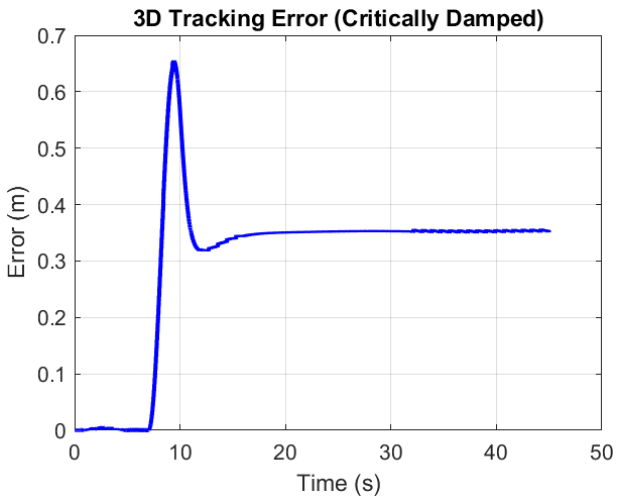
1. **Torque Generation:** Computes the required moments based on the error between desired and actual Euler angles:

$$\tau = K_{p\_att}(\theta_{des} - \theta) + K_{d\_att}(0 - \dot{\theta}) \tag{32}$$

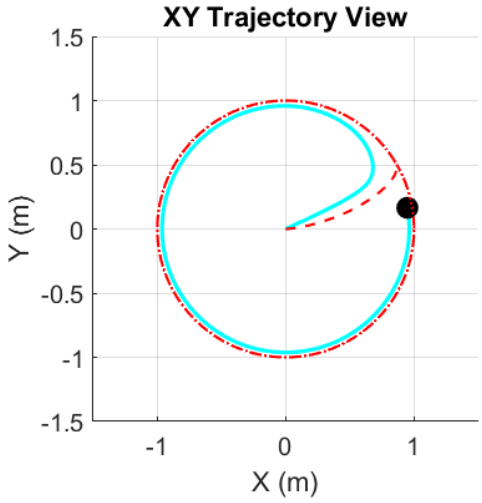
2. **Tilt-Rotor Mixing:** Unlike a standard quadcopter, this system computes a **Thrust Correction Vector** based on the desired roll/pitch. This vector is projected onto the tilt axes of the rotors to determine the physical servo angles required, allowing for vectoring without solely relying on differential thrust.

SIMULATION AND RESULTS

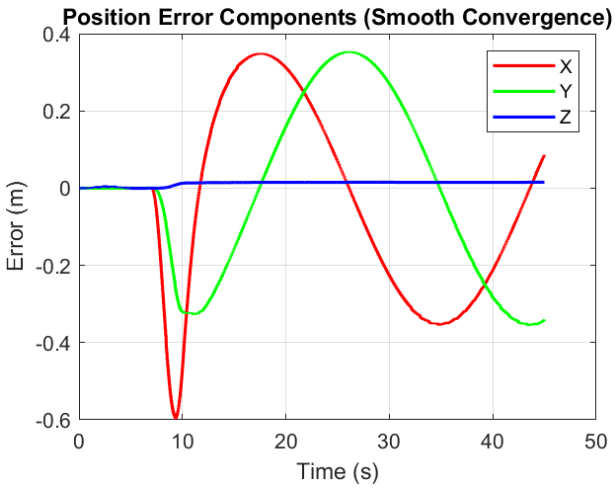
- A MATLAB simulation was done with the developed model to track a circular path and the files are available here: Simulations



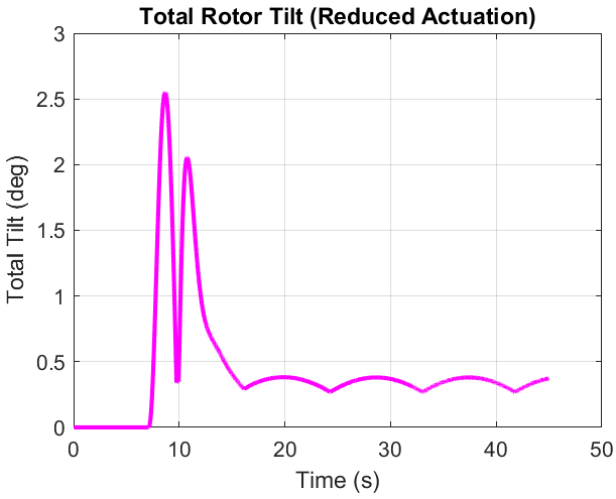
(a) Error in 3D Tracking vs Time



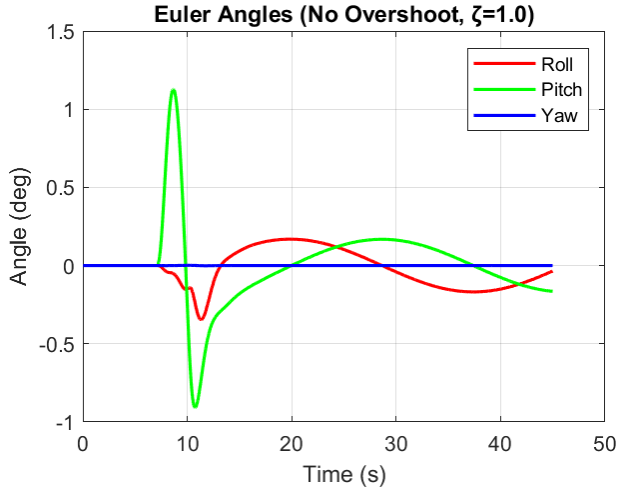
(b) XY Trajectory View



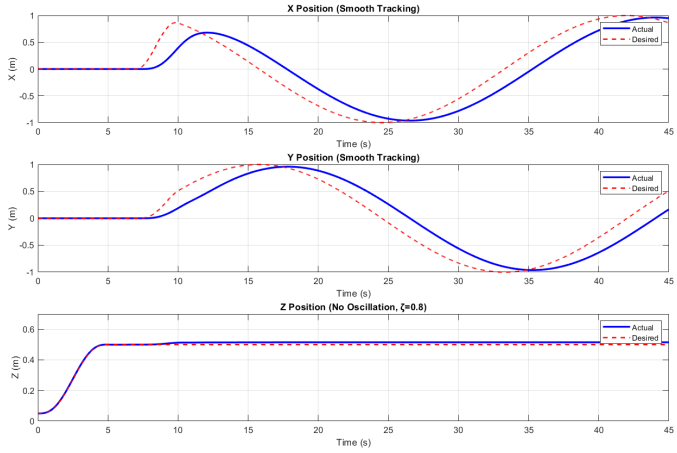
(a) Position Error Components



(b) Total Rotor Tilt vs Time



(a) Euler Angles vs Time



(b) X,Y,Z Position Tracking

FACULTY FEEDBACK AND APPROVAL

Dr. Santhakumar Mohan

29/11/2025

Signature & Date

Dr. Vijay Muralidharan

Progress is good.

29-11-2025

Signature & Date